

SICLAW: Defense-in-Depth Security Architecture for Production LLM-Based SRE Agents

Anonymous submission

Abstract

Large language model (LLM) agents are increasingly deployed for Site Reliability Engineering (SRE)—incident diagnosis, root-cause analysis, and remediation in Kubernetes. We report an emergent-safety finding that reframes how such agents should be deployed: across 400 real diagnosis traces on a live cluster (four LLM brains, *no adversary*), agents *spontaneously* generate sandbox-escape commands—host-namespace entry, container-runtime exec, network egress, and cluster writes—at 0.12 per case, with one brain 3–5× more escape-prone than its peers and the *most accurate* brain among the most unsafe. This behavior is invisible to the accuracy metrics existing benchmarks report, and existing SRE agents (Stratus, FLASH, RCAgent) and benchmarks (AIOpsLab, SREGym, ITBench) provide no security model to contain it. We present SICLAW, the first production-grade SRE agent that treats the LLM as *untrusted code*, enforcing safety through a six-layer defense-in-depth architecture: OS-level dual-user isolation with a novel setgid transitive-privilege mechanism, a six-pass command validator that decouples command definitions from execution-context policies, a three-layer output sanitizer, a four-stage guard pipeline for LLM output unreliability, and an extensible skill architecture with three-gate review. SICLAW runs in three modes—single-user TUI, multi-user local, and production Kubernetes with per-pod isolation—sharing one agent core. SICLAW’s defense neutralizes *every* one of the 0.12/case escape commands at no observed accuracy cost, blocks a 30-attack static red-team (no single layer exceeds 70% coverage), and refuses 35/36 indirect-prompt-injection payloads embedded in live cluster resources with zero exfiltration. Under a real LLM-as-judge checklist oracle it attains an 88–90% diagnosis pass rate (mean 0.898–0.930) across the four brains with no self-judge bias, plus a first feasibility probe on 10 GPU/RDMA hardware-fault scenarios. While general agent-safety benchmarks measure unsafe behavior in emulated sandboxes and privilege-control frameworks propose policy layers, to our knowledge SICLAW is the first to measure emergent unsafety in a *live, production-grade SRE deployment on a real cluster* and to show that an OS-level defense contains it at *zero measured accuracy cost*—establishing agent-as-untrusted-code as an empirical, not hypothetical, requirement for infrastructure agents.

Introduction

The rapid advancement of large language models (LLMs) has sparked significant interest in applying LLM-based

agents to Site Reliability Engineering (SRE) tasks (Chen et al. 2025b,a; Zhang et al. 2024). These agents promise to automate incident diagnosis, root cause analysis (RCA), and remediation in complex cloud-native environments, potentially reducing mean time to resolution (MTTR) and alleviating the chronic shortage of experienced SRE practitioners.

Recent systems show encouraging *capability*. Stratus (Chen et al. 2025a) reaches 78.5% mitigation on the SREGym benchmark (Clark et al. 2026) via multi-agent collaboration with undo-and-retry; FLASH (Zhang et al. 2024) reaches 59.3% on AIOpsLab (Chen et al. 2025b) via workflow decomposition with hindsight; and benchmarks such as AIOpsLab, SREGym, and ITBench (Jha et al. 2025) rigorously measure diagnostic accuracy.

Beneath these capability gains, however, lies a safety problem that accuracy metrics do not surface. Instrumenting a production SRE agent on a live cluster, we find that *even under benign, read-only diagnostic tasks with no adversary*, the agent spontaneously generates commands that would escape its sandbox—host-namespace entry (`nsenter`), container-runtime exec (`crictl`), network egress (`wget/nc`), and cluster writes (`kubectl rollout`)—at 0.12 per case across 400 traces, with one model 3–5× more escape-prone than its peers and the *most accurate* model among the most unsafe (§). Treating the LLM as trusted code is therefore untenable: an unconstrained agent simply executes these commands.

Yet **no existing SRE agent system treats the LLM as untrusted code**. AIOpsLab mentions “security policy filters” for shell execution without specification; SREGym exposes unrestricted `kubectl` access (“cluster control for executing any commands”); Stratus, FLASH, and RCAgent (Wang et al. 2024) provide no documented security model. Combined with susceptibility to prompt injection (Ruan et al. 2024) and hallucination-driven command generation, this exposes production infrastructure to credential theft, data exfiltration, and cluster-wide outages.

We identify five critical gaps in the current SRE agent literature (Table 1):

1. **Agent security:** No *SRE* agent provides defense-in-depth against the agent’s own generated commands; general privilege-control frameworks (Shi et al. 2025) add a policy layer but no OS-level isolation and are not evaluated on live SRE deployments.

Table 1: Capability comparison of SRE agent systems. SICLAW is the only one addressing agent security, multi-tenant isolation, output sanitization, and extensible knowledge.

Capability	AIOpsLab	SREGym	Stratus	FLASH	SICLAW
Live K8s environment	✓	✓	✓	✓ [†]	✓
Agent security model	—	—	—	— [†]	6-layer
Emergent-unsafety meas.	—	—	—	—	0.12/case
Multi-tenant isolation	—	—	—	—	3 modes
Output sanitization	—	—	—	—	3-layer
Extensible skills	—	—	—	fixed	4-tier
Guard pipeline	—	—	—	—	4-stage
GPU/RDMA fault eval.	—	—	—	—	10 cases

[†] Proprietary; not reproducible.

2. **Multi-tenant isolation:** All systems assume single-user, single-agent scenarios.
3. **Output sanitization:** No system prevents sensitive data (credentials, tokens) in command outputs from reaching the LLM’s context.
4. **Agent reliability:** LLM output unreliability (malformed tool calls, context overflow) is documented by every benchmark but addressed by none.
5. **Knowledge extensibility:** Existing systems use fixed tool sets or static Standard Operating Procedures (SOPs), preventing organizational knowledge accumulation.

We present SICLAW, a production-grade SRE agent system that addresses all five gaps. Our key contributions are:

- **Measured emergent unsafety in a live SRE deployment** (§, §): Where prior agent-safety work measures unsafe behavior in emulated sandboxes, we *measure*—across 400 benign diagnosis traces on a real Kubernetes cluster—that competent SRE agents spontaneously emit 0.12 sandbox-escape commands per case, model-dependent (3–5×) and decoupled from accuracy (the most accurate brain is among the most escape-prone). This establishes agent-as-untrusted-code as an empirical necessity for infrastructure agents, not a hypothetical.
- **Defense-in-depth security architecture** (§): Six independent security layers, including a novel dual-user OS isolation model with setgid-based transitive privilege for selective credential access, a six-pass command validation pipeline, and a three-layer output sanitization pipeline. Each layer provides protection independently—compromising one does not disable the others.
- **Guard pipeline for agent reliability** (§): A four-stage message interception architecture (input, context, output, persist) that systematically addresses LLM output unreliability, including malformed tool call repair, context budget enforcement, and stream event correction.
- **Multi-tenant runtime polymorphism** (§): Three runtime modes sharing a unified agent core, with formal

isolation invariants for shared-filesystem and per-pod deployment models.

- **Extensible skill architecture** (§): A four-tier skill hierarchy with three-gate review (static analysis, AI semantic review, human approval) enabling safe organizational knowledge accumulation.
- **Comprehensive evaluation** (§): 100 Kubernetes fault diagnosis cases across 10 categories and *four* LLM brains under a real LLM-as-judge oracle; a *measured* study of emergent agent unsafety and its containment; an indirect-prompt-injection study; a security red-team with layer ablation; and a GPU/RDMA fault-diagnosis feasibility probe.

Related Work

LLM-Based SRE Agents. LLM-based operations agents have advanced rapidly: from LLM root-cause analysis with historical reports (Chen et al. 2024) and query recommendation (Jiang et al. 2024), through tool-augmented ReAct-style RCA (Wang et al. 2024; Yao et al. 2023) and tree-of-thought database diagnosis (Zhou et al. 2024), to workflow decomposition with hindsight (Zhang et al. 2024), multi-agent observe/diagnose/mitigate pipelines (Stratus, highest mitigation on SREGym) (Chen et al. 2025a), SOP-guided multi-agent workflows (Pei et al. 2025), LLM-assisted monitoring setup (Yu et al. 2024b), and multi-modal RCA (Zhang et al. 2025). None of these treats the agent as untrusted or measures its unsafe behavior; all optimize *capability* (diagnosis/mitigation accuracy), an axis orthogonal to the *safety* properties we study.

AIOps Benchmarks. AIOpsLab (Chen et al. 2025b) established the first holistic framework (48 problems over DeathStarBench (Gan et al. 2019) microservices, the Agent-Cloud Interface concept); SREGym (Clark et al. 2026) extended it with noise injection, full-stack faults (including eBPF-simulated hardware failures), and an LLM-as-judge checklist oracle at $\kappa = 0.90$ human agreement; and IT-Bench (Jha et al. 2025), OpenRCA (Xu et al. 2025), and Cloud-OpsBench (Wang et al. 2026) broaden to general IT automation, root-cause localization, and reproducible agentic RCA. These benchmarks evaluate diagnostic capability but assess neither agent safety, multi-tenant scenarios, nor GPU/RDMA faults—despite GPU hardware errors being the dominant failure mode in production AI clusters (Cui et al. 2025; Wan et al. 2025).

Agent Safety and Privilege Control. A parallel line of work secures *general* tool-using agents: AgentBench (Liu et al. 2024) and ToolEmu (Ruan et al. 2024) evaluate agents in sandboxed or emulated environments; Progent (Shi et al. 2025) enforces least privilege through a programmable tool-call policy language; and KubeIntellect (Thota 2025) wraps a Kubernetes-management agent in RBAC and sandboxing. These contribute valuable *policy* layers, but operate over general or emulated agents, add no *OS-level* isolation beneath the policy, and do not report the capability cost of their controls. SICLAW differs on all three: it adds an OS-level isolation primitive (setgid transitive privilege) beneath

the validator, runs as a *live, read-only* production SRE agent, and quantifies that its defense costs *zero* diagnostic accuracy (§).

Measuring Agent Unsafety and Injection. A second line measures *when* agents act unsafely: RedCode (Guo et al. 2024) benchmarks risky code execution, SandboxEscapeBench (UK AI Security Institute et al. 2026) measures container breakout, and Saber (Bhatt et al. 2026) probes operational-safety failures including benign tasks that invite unsafe shortcuts—the closest prior art to our emergent-unsafety finding. Indirect-prompt-injection suites—InjecAgent (Zhan et al. 2024) and AgentDojo (Debenedetti et al. 2024)—measure injection susceptibility in tool-integrated agents. All of these run in *emulated or static-benchmark* settings over general or coding agents. SICLAW instead measures emergent unsafety in a *live read-only SRE deployment on a real Kubernetes cluster* and—uniquely among this group—reports the *diagnostic-accuracy cost* of containment, which we measure to be zero; our indirect-injection probe (§) is complementary to InjecAgent and AgentDojo, which an infrastructure-specific suite at their scale should extend. To our knowledge, no prior system both measures this behavior on production infrastructure and contains it with an OS-level defense at no accuracy cost.

Threat Model and Problem Formulation

Core Assumption. We model the LLM agent as *untrusted code* that generates shell commands executing in a container with Kubernetes cluster access. The agent may attempt—through prompt injection, hallucination, or adversarial manipulation—to: (T1) *read credentials* (kubeconfig, mTLS certificates, API keys); (T2) *exfiltrate data* to external endpoints; (T3) *escalate privileges* beyond read-only Kubernetes operations; (T4) *leak sensitive information* through command outputs that enter the LLM context.

Trust Boundaries. The AgentBox container holds two trust domains: the *main process* (Node.js runtime, LLM API client, tool validation) as user `agentbox`, and *child processes* (LLM-generated shell commands) as user `sandbox`. The boundary lies at the command-execution interface: the main process validates and sanitizes; child processes execute with minimal privilege.

Security Goals. We enforce three security properties: (1) **credential isolation**—child processes cannot read credential files even if command validation is bypassed; (2) **least privilege**—only whitelisted commands with approved flags execute, with `kubectl` restricted to 13 read-only subcommands; and (3) **output safety**—sensitive data (tokens, keys, connection strings) in command outputs is redacted before entering the LLM context.

Problem Formulation. At each step an agent generates a command c_t from history and observations; a security pipeline $\mathcal{S}$ approves, modifies, or blocks it ($c'_t = \mathcal{S}(c_t)$), and the resulting output is sanitized ($r'_t = \mathcal{O}(r_t, c'_t)$) before re-entering the context, with a guard pipeline $\mathcal{G}$ repairing malformed messages and enforcing context budgets. The goal is

to maximize diagnostic accuracy subject to $\mathcal{S}$ enforcing credential isolation and least privilege and $\mathcal{O}$ enforcing output safety at every step.

System Architecture

SICLAW is built on a single brain abstraction (OpenAI-compatible provider interface) with a unified tool registry, guard pipeline, and security architecture. As Figure 1 shows, user intent flows through a control plane that orchestrates per-user *AgentBox* sessions, each investigating read-only infrastructure targets; this section details the agent core and its security architecture (Figure 2).

Defense-in-Depth Security Architecture

The security architecture consists of six independent layers, each providing protection even if others are compromised. This section details the three most novel layers.

Layer 1: OS-Level Dual-User Isolation The AgentBox container contains two Linux users: `agentbox` (UID 1000, groups: `agentbox`, `kubecred`) runs the main Node.js process, and `sandbox` (UID 1001, group: `sandbox`) runs all LLM-generated shell commands via `sudo -E -u sandbox`.

Transitive Privilege via `setgid`. `kubectl` must read kubeconfig files, but `sandbox` must not. We solve this with a `setgid` mechanism: `kubectl` is installed `setgid` to the `kubecred` group (`-rwxr-sr-x root:kubecred`) and kubeconfig is `agentbox:kubecred mode 0640`. When `sandbox` runs `kubectl`, the `setgid` bit temporarily grants `kubecred` membership so `kubectl` alone reads the kubeconfig; other commands (`grep`, `cat`, `jq`) lack it. This preserves *pipeline compatibility*—`kubectl get pods | grep Error` works because data flows through non-access-controlled stdout while credentials stay protected by filesystem permissions. Since kubeconfig (0640) and mTLS certs/API keys (0600) are unreadable by `sandbox`, even a full Layer 2 bypass leaves credential access independently blocked.

Layer 2: Six-Pass Command Validation Pipeline Every LLM-generated command passes through six independent validation passes before execution, over a context $\xi \in \{\text{local}, \text{node}, \text{pod}\}$: (1) *shell operators*—block `$()`, backticks, `<()/>`, redirections, and embedded newlines; (2) *pipeline extraction*—parse the command into stages $[s_1|s_2|\dots]$; then, per stage, (3) *binary whitelist*—verify $\text{binary}(s_i) \in \mathcal{W}_\xi$ (the context-specific allowlist); (4) *subcommand validation*—for `kubectl`, restrict to 13 read-only subcommands (`get`, `describe`, `logs`, ...); (5) *flag restrictions*—apply per-command rules (`pipeOnly`, `noFilePaths`, `blockedFlags`, `allowedFlags`); and finally (6) *sensitive paths*—block commands targeting credential/configuration files.

Context-Policy Decoupling. A key design innovation separates *command definitions* (COMMANDS registry: binary names, categories, per-command rules) from *context policies* (CONTEXT.POLICIES: which categories each execution context allows). For example, the `local` context blocks

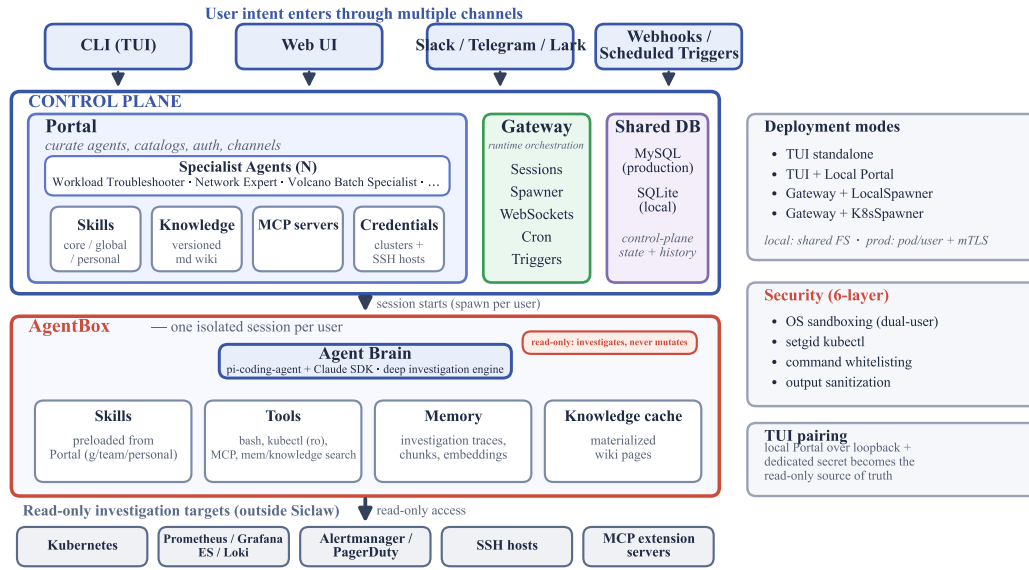


Figure 1: SICLAW system overview. User intent enters through four channels and reaches the *control plane* (Portal, Gateway, shared DB), which curates specialist agents, skills, knowledge, MCP servers, and credentials. Each session spawns an isolated *AgentBox*—a Kubernetes pod in production, in-process locally, or embedded in a standalone TUI—holding the agent brain, preloaded skills, read-only tools, a private memory DB, and a materialized knowledge cache. Every target it touches is accessed *read-only*, and a six-layer security architecture (Figure 2) constrains all agent actions.

file-reading commands (`cat`, `ls`, `find`)—the agent has dedicated file tools with path access control—while the pod context allows them. This keeps the ~80-binary allowlist maintainable: a new command specifies its category and rules once, and context policies determine where it may execute.

Layer 3: Three-Layer Output Sanitization Command outputs may carry sensitive data (JWT tokens, connection strings, PEM certificates) that would enter the LLM context if not intercepted. Three layers address this: (1) *pre-execution analysis* (`analyzeOutput`) inspects the command to pick a sanitizer; (2) *post-execution redaction* (`applySanitizer`) redacts sensitive env-var names (`*API_KEY`, `*_SECRET`, `DATABASE_URL`) and value patterns (JWT `eyJ...`, PEM `-----BEGIN`, connection strings); and (3) a *pipeline fallback* applies line-level pattern redaction to the final output when a downstream stage (e.g. `jq` in `kubectl get secret -o yaml | jq .data`) has no registered rule, catching what the first two missed.

Layers 4–6: Container Hardening, File I/O, Environment **Layer 4 (Container hardening)** drops all Linux capabilities except the five required for the dual-user model (`SETUID`, `SETGID`, `CHOWN`, `FOWNER`, `AUDIT_WRITE`), uses a read-only root filesystem, and disables the service-account token mount. **Layer 5 (File I/O restrictions)** has the agent’s file tools enforce `assertPathAllowed()`, scoping reads/writes to designated directories so the agent cannot reach credential files through its own tools. **Layer 6 (Environment sanitization)** runs `sanitizeEnv()`

before spawning child processes, stripping API keys, tokens, and sensitive variables to prevent leakage via `/proc/{pid}/environ`.

The security architecture is validated by 184 passing unit tests (97 command validator, 48 output sanitizer, 15 pipeline integration, 16 guard pipeline, 8 tool-call repair).

Guard Pipeline for Agent Reliability

LLM agents exhibit systematic output unreliability—malformed tool-call JSON, missing tool-call IDs, context-window overflow from large outputs, corrupted streaming events—that every benchmark documents (Chen et al. 2025b; Clark et al. 2026) yet none systematically solves. The guard pipeline intercepts agent messages at four life-cycle stages. *Input guards* transform the message history before each API call: dropping tool-call blocks missing required fields (`id/name/input`) and ensuring every `toolCall` has a matching `toolResult` (inserting synthetic error results where needed). *Context guards* enforce a three-tier budget: truncate any single tool result over 50% of the window, compact oldest results past 75%, and raise a preemptive overflow signal at 90% before the API rejects the request. *Output guards* repair streaming events in-flight: trimming/ID-filling unnamed tool calls and extracting balanced JSON from partial `toolcall_delta` events by bracket counting. *Persist guards* validate messages before session-history writes: sanitizing tool calls, tracking call/result pairing, inserting synthetic results for orphaned calls, and truncating oversized (>400KB) results. The pipeline uses a *single-wrapper-hook* architecture—each of the brain’s three hook points

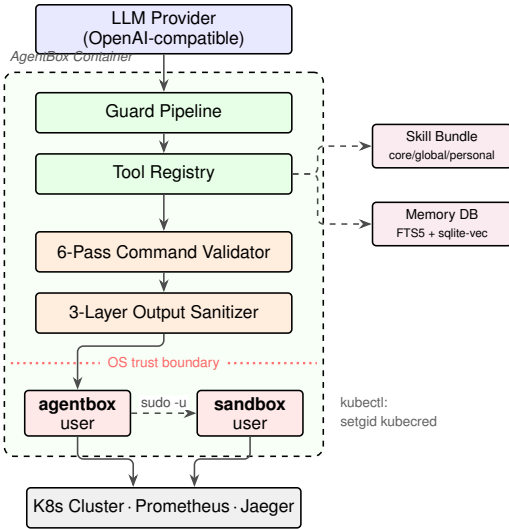


Figure 2: SICLAW architecture. LLM-generated commands flow through the guard pipeline, tool registry, 6-pass validator, and 3-layer sanitizer before crossing the OS trust boundary. Child processes run as the unprivileged sandbox user; `kubectl` reaches the kubeconfig via `setgid`.

(`streamFn`, `appendMessage`, `transformContext`) is wrapped exactly once—avoiding nested monkey-patching while keeping per-stage control.

Multi-Tenant Runtime Polymorphism

As shown in Figure 1, SICLAW supports four runtime modes sharing one agent core, trading isolation for convenience: (1) **standalone TUI**; (2) **TUI + local Portal** (read-only Portal snapshot over loopback with three-gate authentication); (3) **Gateway + LocalSpawner** for multi-user local development on a shared filesystem; and (4) **Gateway + K8sSpawner** for production, with per-user pod isolation and mTLS certificates encoding `userId/workspaceId`. Two invariants matter: on the shared filesystem (mode 3), `materialize()` must never be called (it would overwrite all users’ skill directories), and in production (mode 4) per-pod isolation is the multi-tenant boundary. All modes implement the `BoxSpawner` interface; the unified agent core operates identically across them.

Extensible Skill Architecture

Existing SRE agents use fixed tool sets (Chen et al. 2025b; Clark et al. 2026) or static SOPs (Pei et al. 2025). SICLAW instead organizes diagnostic skills into a four-tier hierarchy ordered by user proximity (personal > skillset > global > core): **core** skills are baked into the container image (versioned with code), **global** are organization-wide, **skillset** are team-scoped (development only, promoted to global for production), and **personal** are per-user. The bundle contract (`buildSkillBundle`) transmits only non-core skills, as core skills exist on disk.

Three-Gate Review. Because skills execute as code, new skills pass three independent safety gates before production:

(1) *static analysis* against 22 danger patterns (shell injection, credential access, unsafe operations); (2) *AI semantic review* of skill intent; and (3) *human approval*. Unapproved skills cannot execute regardless of tier—enabling safe organizational knowledge accumulation, unlike the fixed tool sets of prior systems.

Hybrid Memory System

Each AgentBox maintains a private memory database (SQLite with FTS5 and `sqlite-vec`) holding structured investigation records over a full lifecycle (create → update → resolve → reference). Search blends vector similarity and keyword matching, $\text{score} = w_v \cos(\mathbf{q}, \mathbf{d}) + w_f \text{BM25}(q, d) + w_t \text{decay}(t)$ ($w_v=0.70$, $w_f=0.30$, temporal decay), reranked via Maximal Marginal Relevance; knowledge documents are heading-chunked (~400 tokens), embedded, and dual-indexed, letting the agent recall prior investigations rather than starting each incident cold.

Evaluation

We evaluate SICLAW along five axes: (1) diagnostic capability on 100 Kubernetes faults across *four* LLM brains under a real LLM checklist oracle; (2) *measured* emergent unsafety—how often a benign agent spontaneously emits sandbox-escape commands—and its containment; (3) indirect prompt injection in live cluster resources; (4) a static security red-team with per-layer ablation; and (5) a 10-scenario GPU/RDMA feasibility probe.

Diagnostic Evaluation

Setup We construct a benchmark of 100 Kubernetes fault diagnosis cases across 10 categories (per-category counts in the technical appendix), following the methodology of SRE-Gym (Clark et al. 2026) and AIOpsLab (Chen et al. 2025b), deployed on a real 368-node cluster (2 GPU nodes, H100 NVLink 80GB; Volcano scheduling; Calico NetworkPolicy). Each case injects a specific fault and gives the agent a symptom-level prompt (e.g. “Pod `c031-pending` is stuck in Pending”); the agent investigates autonomously with `kubectl`, `grep`, `jq`, and other whitelisted tools. Every case is run by *four* diverse brains—Claude Sonnet 4.6, Kimi K2.5, DeepSeek V4-Flash, Qwen3.6-27B (400 runs total)—testing generalization beyond a single model. The agent is *read-only*: no mutations are permitted, itself a safety property of the architecture (§), not merely an evaluation convenience.

Scoring Following SRE-Gym’s checklist methodology, each diagnosis is scored 0–1 across five dimensions—**localization** (right target resource), **mechanism** (root cause, not symptom), **scope** (blast radius), **evidence** (concrete observables), and **remediation** (safe, root-cause-matched fix)—by a real LLM checklist judge (Claude Sonnet 4.6) that answers grounded Yes/No questions, each requiring a verbatim trace quote and a confidence label, at temperature 0 against per-case oracle ground truth; a case passes if its mean exceeds 0.65. This judge is deliberately distinct from brittle label matching: re-scoring traces, it reverses artifacts in

Table 2: Diagnostic results across four brains (100 cases each, real LLM checklist judge). The frozen Claude judge ranks DeepSeek, Qwen, and Kimi at or above Claude itself—evidence against self-judge bias.

Brain	Pass%	Score	Loc	Mech	Scope	Evid	Rem
Claude S4.6	88.0	0.898	0.92	0.90	0.86	0.93	0.89
Kimi K2.5	88.0	0.906	0.90	0.88	0.92	0.95	0.89
DeepSeek V4	90.0	0.930	0.94	0.90	0.94	0.96	0.92
Qwen3.6-27B	89.0	0.901	0.91	0.87	0.90	0.95	0.88

both directions—a false pass where the agent called a pod “healthy” yet missed an anti-affinity conflict (0.80 $\rightarrow$ 0.10) and a false failure where a correct diagnosis used non-matching wording (0.47 $\rightarrow$ 1.00). The same frozen judge scores all four brains, avoiding self-judge bias; as a reliability check, an *independent* judge (DeepSeek V4) re-scores a stratified 20-case sample and reproduces its per-question verdicts at Cohen’s $\kappa = 0.87$ (94.5% agreement). The verbatim checklist and agreement protocol are in the technical appendix (Diagnosis Evaluation Checklist and Inter-Judge Agreement); full human- κ validation is the remaining, stronger step.

Results SICLAW achieves an 88–90% pass rate (mean score 0.898–0.930) across the four brains (Table 2). The result generalizes: the frozen Claude judge ranks DeepSeek, Qwen, and Kimi at or above Claude itself, so the headline is neither a single-model artifact nor self-judging. Tool budgets differ sharply (DeepSeek/Kimi resolve most cases in 4.9–6.0 calls and 19–30 s vs. Claude’s 7.4 and 51 s) with no accuracy penalty, and scores are balanced across the five dimensions—scope (0.86–0.94) and remediation (0.88–0.92) stay in range, not the degenerate extremes (scope 0.48, remediation 1.000) that brittle label matching produced.

The accuracy gap lives in *categories*, not dimensions: easy single-fault categories (image-pull, config, storage) saturate at 100%, while the three hardest—network-DNS (57.5% pass), compound multi-fault (64.3%), controller (71.9%)—are weakest *consistently* across brains (full per-category breakdown in the technical appendix). These demand reasoning over multi-resource impact chains (NetworkPolicy+DNS, owner-reference hierarchies); difficulty is intrinsic, not a single-model weakness, and even non-monotonic—nominally “medium” network/controller cases (70.8%) are *harder* than “hard” GPU/compound cases (81.0%). We report these honest sub-threshold cells rather than the saturated aggregate.

Security Red-Team Evaluation

We design 30 attacks across the four threat categories—T1 credential reading (8), T2 exfiltration (8), T3 privilege escalation (8), T4 output leakage (6)—each run as a concrete command string against the real security-pipeline functions (validateCommand, validateShellOperators, postExecSecurity), attributed to the first layer that intercepts it. This *static* suite complements the *behavioral* measurements that follow (emergent unsafety and indirect

Table 3: Emergent unsafety over 400 *benign* traces (no adversary): 47 spontaneously-generated sandbox-escape commands blocked. DeepSeek—the *most accurate* brain—is 3–5 $\times$ more escape-prone, a divergence invisible to accuracy metrics.

Brain	Tool calls	Blocked	Cases	Per case
Claude S4.6	739	8	4	0.08
Kimi K2.5	486	5	5	0.05
DeepSeek V4-Flash	605	27	16	0.27
Qwen3.6-27B	432	7	6	0.07
Total	2262	47	31	0.12

injection). **All 30 attacks are blocked**, with blocks spread across layers (T1 by L2/L4/L6, T2 by L2/L1/L5, T3 by L5/L2, T4 by L4); the per-category block-to-layer map is in the technical appendix (Security Red-Team Catalog). Since this tests attacks the pipeline was designed to catch, the load-bearing evidence is behavioral, and an adaptive/LLM-generated red-team is future work.

Security Layer Ablation

To validate the defense-in-depth design, we test each security layer independently—“if *only* this layer existed, how many of the 30 attacks would it block?” (per-layer coverage table in the technical appendix, Security Red-Team Catalog). Three findings hold: (1) no single layer exceeds 70% coverage (L2’s binary whitelist is the strongest at 21/30, but misses all kubectl-write attacks, caught by L5, and output leakage, caught by L4); (2) 18/30 attacks (60%) are caught by 2+ layers, demonstrating true defense-in-depth; and (3) 12 attacks are caught by exactly one layer, so removing it would open a vulnerability (*e.g.* kubectl-write commands are caught *only* by L5’s subcommand check).

Emergent Unsafety During Benign Diagnosis

Our central safety claim is not about adversaries: it is that a *benign* agent, given only a read-only diagnostic task, spontaneously emits commands that would escape its sandbox. We measure this directly. Across all 400 diagnosis traces (4 brains $\times$ 100 cases, 2,262 tool calls, *no* injected adversary), the security pipeline blocked **47 sandbox-escape commands—0.12 per case** (Table 3). These are pure hallucination, not attacks: nsenter into host namespaces, crictl exec/pull, wget/nc egress, xargs/python3/awk shell-outs, \$() substitution, input redirection, and kubectl rollout. 46 of the 47 are caught by the L2 binary/subcommand whitelist; the remaining one by the sensitive-path rules. The operational definition of a sandbox-escape command and the full counting protocol are in the technical appendix (Emergent-Unsafety Measurement Protocol).

The rate is strongly model-dependent: DeepSeek emits escape commands **3–5 $\times$ more often** than its peers (0.27/case vs. 0.05–0.08)—yet is simultaneously the most accurate brain (Table 2), so this safety-relevant gap is *invisible* to

accuracy metrics. This is the empirical core of the agent-as-untrusted-code thesis: unsafe command generation is a baseline property of competent agents under normal operation, not an adversarial edge case, varying by model in ways accuracy benchmarks miss.

Accuracy cost. Does blocking these commands hurt diagnosis? Examining every failed case for a security-attributable failure—a block whose output a correct diagnosis would have required—we found *none*: all 47 are redundant escape attempts (e.g. `wget` to a service already reachable via `kubectl get/describe`), and every diagnosis completed using only the read-only whitelist (`get`, `describe`, `logs`, `events`, `top`). To confirm this, we ran a controlled security-off A/B on DeepSeek—the most escape-prone brain—re-running all 100 cases with the read-only analysis toolkit widened (blocked commands 27 → 19; irreversible actions still blocked). Accuracy is statistically unchanged: pass rate 90.0% → 88.0%, mean score 0.930 → 0.919 ($\Delta = -0.011$), 83/100 cases identical and the rest splitting symmetrically (8 better, 9 worse)—noise, not a security penalty. We thus observe **no measurable accuracy cost** (A/B protocol and preserved irreversible-action “safety floor” in the technical appendix).

Indirect Prompt Injection via Tool Output

A distinctive infrastructure threat is *indirect* prompt injection: a malicious instruction planted in cluster state (a ConfigMap value, pod-log line, or annotation) that the agent ingests as tool output while investigating. We embed 12 such payloads—service-account-token reads, `curl` exfiltration, RBAC escalation, reverse shells, secret dumps—in live cluster resources and run each against 3 brains (36 trials; Claude omitted due to an unrelated gateway incompatibility). **35 of 36 are refused outright**: the agent recognizes the injected instruction and declines to act. In the one case where DeepSeek attempted an embedded token read, the sensitive-path layer neutralized it; **zero payloads exfiltrated data**. Multi-turn chains and payloads disguised as legitimate runbook steps are future work.

GPU/RDMA Fault Diagnosis: A Feasibility Probe

No published SRE benchmark evaluates agent diagnosis of GPU hardware errors (Xid events, ECC failures, NVLink degradation) or RDMA fabric issues (link flapping, PFC storms, QP errors) (Clark et al. 2026; Chen et al. 2025b; Yu et al. 2024a), despite these being the dominant failure modes in production GPU clusters. We design 10 scenarios by *observable-signal simulation*: rather than injecting hardware faults (which needs physical access), we reproduce the signals an SRE agent would observe—`dmesg`, `nvidia-smi`, RDMA counters (`ibstat`, `ethtool`), NCCL logs—as Kubernetes ConfigMaps, matching the agent’s real interface. Lacking oracle ground truth for these synthetic scenarios, we score them by a heuristic signal-coverage rubric (did the diagnosis name the Xid code, affected GPU, link-layer counter?)—*not* the LLM checklist judge—grounded in production fault data (Cui et al. 2025; Wan et al. 2025), so we present this as a feasibility probe, not a graded benchmark.

All 10 scenarios reach the pass threshold (mean rubric 0.923; per-case breakdown in the technical appendix). Qualitatively, the agent maps Xid codes to specific GPUs, distinguishes thermal throttling from genuine OOM, and attributes RDMA faults to link-layer counters; in compound case g09 it names *both* independent root causes (GPU ECC + RDMA congestion). We read this as preliminary evidence that an LLM agent *can* diagnose GPU/RDMA faults given appropriate telemetry, while cautioning that the heuristic rubric and simulated signals make this weaker evidence than the LLM-judged Kubernetes results.

Comparison and scope. Comparison with SREGym is not apples-to-apples: it reports *end-to-end success including mitigation* (Claude Code 60.7%, Stratus 54.8% (Clark et al. 2026)), whereas SICLAW is graded read-only on its own diagnosis-only cases. We make no capability claim against SREGym; our contribution is the orthogonal *safety* dimension those benchmarks omit, and running SICLAW directly on SREGym’s cases is the clear next step.

Discussion and Conclusion

SICLAW is, to our knowledge, the first production-grade SRE agent that treats the LLM as untrusted code, and our central finding makes that stance empirical: across 400 benign traces, competent agents spontaneously emit 0.12 sandbox-escape commands per case through ordinary hallucination—no adversary, model-dependent (3–5×), decoupled from accuracy. The six-layer defense neutralizes all 47 escapes and resists 35/36 indirect-injection payloads with zero exfiltration, at no measured accuracy cost (90.0% → 88.0%); since unsafe command generation is a *measured* baseline property and not an edge case, we argue agent-as-untrusted-code is necessary for production. The architecture is not Kubernetes-specific—the dual-user model, validator, and sanitizer apply to any LLM agent executing shell commands, and context-policy decoupling adds new contexts (cloud CLIs, database tools) without touching command definitions—and the GPU/RDMA probe suggests LLM agents *can*, given telemetry, diagnose a fault class absent from existing SRE benchmarks (Cui et al. 2025; Yu et al. 2024a), motivating a labelled, LLM-judge-graded GPU/RDMA benchmark on real hardware.

Limitations. The evaluation uses a single cluster topology and a single run per case (no confidence intervals); cross-cluster and multi-seed runs are future work. The LLM judge is corroborated by an independent judge ($\kappa = 0.87$) but not yet by human labels (the SREGym standard; in progress). The “no accuracy cost” A/B is on a single brain (DeepSeek) against a pre-existing security-on run, so run-to-run noise is not fully removed. The GPU/RDMA probe uses simulated signals and a heuristic rubric; the static red-team covers attacks the pipeline was built to catch, not adaptive or multi-turn injection; and the Stratus/Claude Code comparison relies on published results, not direct reproduction. We hope SICLAW provides both a foundation and a research agenda for secure autonomous operations.

References

- Bhatt, M.; et al. 2026. Saber: Benchmarking Operational Safety of LLM Coding Agents in Stateful Project Workspaces. *arXiv preprint arXiv:2606.01317*.
- Chen, Y.; Pan, J.; Clark, J.; Su, Y.; Zheutlin, N.; Bhavya, B.; Arora, R.; Deng, Y.; Jha, S.; and Xu, T. 2025a. Stratus: A Multi-agent System for Autonomous Reliability Engineering of Modern Clouds. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Chen, Y.; Shetty, M.; Somashekar, G.; Ma, M.; Simmhan, Y.; Mace, J.; Bansal, C.; Wang, R.; and Rajmohan, S. 2025b. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *Proceedings of Machine Learning and Systems (MLSys)*.
- Chen, Y.; Xie, H.; Ma, M.; Kang, Y.; Gao, X.; Shi, L.; Cao, Y.; Gao, X.; Fan, H.; Wen, M.; et al. 2024. RCACopilot: Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proceedings of the European Conference on Computer Systems (EuroSys)*.
- Clark, J.; Su, Y.; Pial, S. M. R.; Tian, Y.; Gniedziejko, L.; Jacobsen, H.-A.; Chen, Y.; and Xu, T. 2026. SREGym: A Live Benchmark for AI SRE Agents with High-Fidelity Failure Scenarios. *arXiv preprint arXiv:2506.XXXXX*.
- Cui, R.; et al. 2025. Characterizing GPU Resilience and Impact on AI/HPC Systems. *arXiv preprint arXiv:2503.11901*.
- Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- Gan, Y.; Zhang, Y.; Cheng, D.; Shetty, A.; Rath, P.; Katarki, N.; Bruno, A.; Hu, J.; Ritchken, B.; Jackson, B.; et al. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- Guo, C.; et al. 2024. RedCode: Risky Code Execution and Generation Benchmark for Code Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- Jha, S.; Arora, R. R.; Watanabe, Y.; Yanagawa, T.; Chen, Y.; Clark, J.; et al. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- Jiang, Y.; Zhang, C.; He, S.; Yang, Z.; Ma, M.; Qin, S.; Kang, Y.; Dang, Y.; Rajmohan, S.; Lin, Q.; and Zhang, D. 2024. Xpert: Empowering Incident Management with Query Recommendations via LLMs. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR)*.
- Pei, C.; Wang, Z.; Liu, F.; Li, Z.; Liu, Y.; He, X.; Kang, R.; Zhang, T.; Chen, J.; Li, J.; Xie, G.; and Pei, D. 2025. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In *Companion Proceedings of the Web Conference (WWW)*.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *International Conference on Learning Representations (ICLR)*.
- Shi, T.; et al. 2025. Progent: Programmable Privilege Control for LLM Agents. *arXiv preprint arXiv:2504.11703*.
- Thota, S. 2025. KubeIntellect: A Modular LLM-Orchestrated Agent Framework for End-to-End Kubernetes Management. *arXiv preprint arXiv:2509.02449*.
- UK AI Security Institute; et al. 2026. Quantifying Frontier LLM Capabilities for Container Sandbox Escape. *arXiv preprint arXiv:2603.02277*.
- Wan, M.; et al. 2025. ByteRobust: Robust LLM Training Infrastructure. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*.
- Wang, Y.; Yu, G.; Huang, H.; Wang, Z.; Huang, Y.; Chen, P.; and Lyu, M. R. 2026. Cloud-OpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems. *arXiv preprint arXiv:2602.XXXXX*.
- Wang, Z.; Liu, Z.; Zhang, Y.; Zhong, A.; Fan, J.; Wu, L.; and Wen, Q. 2024. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented LLMs. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*.
- Xu, J.; Zhang, Q.; Zhong, Z.; He, S.; Zhang, C.; Lin, Q.; Pei, D.; He, P.; Zhang, D.; and Zhang, Q. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures? In *International Conference on Learning Representations (ICLR)*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.
- Yu, G.; et al. 2024a. A Survey on Failure Analysis and Fault Injection in AI Systems. *ACM Transactions on Software Engineering and Methodology*.
- Yu, Z.; Ma, M.; Zhang, C.; Qin, S.; Kang, Y.; Bansal, C.; Rajmohan, S.; Dang, Y.; Pei, C.; Pei, D.; Lin, Q.; and Zhang, D. 2024b. MonitorAssistant: Simplifying Cloud Service Monitoring via Large Language Models. In *Companion Proceedings of the International Symposium on Foundations of Software Engineering (FSE)*.
- Zhan, Q.; Liang, Z.; Ying, Z.; and Kang, D. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics (ACL)*.
- Zhang, X.; Mittal, T.; Bansal, C.; Wang, R.; Ma, M.; Ren, Z.; Huang, H.; and Rajmohan, S. 2024. FLASH: A Workflow Automation Agent for Diagnosing Recurring Incidents. *arXiv preprint arXiv:2410.XXXXX*.

Zhang, X.; Wang, Q.; Li, M.; Yuan, Y.; Xiao, M.; Zhuang, F.; and Yu, D. 2025. TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent With Multi-Modality Observation Data. *IEEE Transactions on Services Computing*, 18(6).

Zhou, X.; et al. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proceedings of the VLDB Endowment*.

The accompanying Technical Appendix (supplementary material) provides the additional detail needed to reproduce every experiment in §: the model and cluster configuration, how the 100-case benchmark is constructed, the verbatim LLM-judge rubric and inter-judge agreement protocol, the operational definition and counting protocol behind the emergent-unsafety result, the security-off A/B protocol, the red-team catalog and layer map, the indirect-injection payloads, the GPU/RDMA probe, and worked case studies. The AAAI reproducibility checklist below stays in the main paper.

Reproducibility Checklist

Self-reported per the AAAI-26 reproducibility checklist. Authors should re-verify every answer (especially code/data release) against the final artifact before submission.

This paper. (1) Conceptual outline/pseudocode of methods introduced: **yes** (§–§, and the technical appendix protocols for emergent-unsafety measurement through the security red-team catalog). (2) Opinions/hypotheses delineated from facts: **yes** (feasibility probes and single-run results are explicitly flagged). (3) Pedagogical references for background: **yes**.

Theoretical contributions. The paper’s contributions are primarily systems and empirical; the only formal claims are the multi-tenant isolation invariants (§), stated as operational contracts. (1) Assumptions stated: **partial**. (2) Claims stated formally: **partial**. (3) Proofs included: **no** (argued, not proved). (4) Proof sketches/intuitions: **partial**. (5) Citations to tools: **yes**. (6) Claims demonstrated empirically: **yes**. (7) Code to disprove claims: **NA**.

Datasets. (1) Motivation for datasets: **yes** (technical appendix, Benchmark Construction). (2) Novel datasets in a data appendix: **partial** (described in the technical appendix, Benchmark Construction; case manifests in the code release). (3) Public on publication: **partial** (intended, with a responsible-release note for the attack/injection catalogs—authors to confirm). (4) Existing datasets cited: **yes** (SRE-Gym, AIOpsLab, DeathStarBench). (5) Existing datasets public: **yes**. (6) Non-public datasets described: **yes** (cluster and cases; technical appendix, Model and Infrastructure Configuration and Benchmark Construction).

Computational experiments. (1) Hyperparameter ranges/selection: **partial** (judge $\tau = 0.65$, temperature 0 documented; few tunable parameters). (2) Pre-processing code: **partial**. (3) Source code in a code appendix: **partial** (system and harness released). (4) Code public on publication: **partial** (authors to confirm). (5) Code commented with paper references: **partial**. (6) Random-seed method: **partial** (judge is deterministic at temperature 0; agent sampling is provider-default and single-run per case—a stated limitation). (7) Computing infrastructure specified: **yes** (technical appendix, Model and Infrastructure Configuration). (8) Evaluation metrics formally described: **yes** (technical appendix, Diagnosis Evaluation Checklist and Inter-Judge Agreement). (9) Number of runs per result: **yes** (one

run per case, disclosed). (10) Beyond single-dimensional summaries: **partial** (per-category/-dimension/-brain breakdowns and inter-judge κ ; no confidence intervals—a stated limitation). (11) Statistical-significance tests: **no** (the A/B and category analyses are descriptive; a paired significance test is future work). (12) Final hyperparameters listed: **yes** (technical appendix, Model and Infrastructure Configuration).